Two Failure Modes in Categorical Target-Leakage Screening: Sparsity Bias and Complete-Case Deletion

Jaime Cano Moraño
Illinois Institute of Technology
USA
jcanomorano@hawk.iit.edu

ABSTRACT

Categorical target-leakage screening typically thresholds Cramér's V. We show two independent failure modes on the same statistic, each traceable to a standard tooling default rather than to V itself. *Sparsity bias*: the Titanic `name` column scores $V = 0.999$ against survival despite carrying no information, because 1,305 of 1,307 names are singleton categories; a bias correction [2] collapses this to $\widetilde{V} = 0.000$, while two well-behaved low-cardinality columns barely move (`sex`: 0.529⟶0.528; `embarked`: 0.184⟶0.180). *Complete-case deletion*: `pandas.crosstab`, and the PPS-based association used by ydata-profiling and Deepchecks' `FeatureLabelCorrelation`, all drop rows with missing feature values before testing association—the default nearly everywhere. On Titanic's `boat` column (a post-hoc leak, 62.9% missing, informative missingness), this drops the measured association from $V = 0.948$ (full data) to $V = 0.420$ (non-missing subset only), hiding the leak. We fix this in our own pipeline—encoding missingness as an explicit category, matching how mutual information already treats it—recovering $V = 0.951$ and raising our unified score from $\mathcal{L} = 0.739$ (warning) to $\mathcal{L} = 0.925$ (error). A negative replication on NSL-KDD (125,973 rows; three categorical features spanning cardinality 3–70, no missing values) confirms the sparsity mechanism requires genuine per-category sparsity, not cardinality alone: raw and corrected V are numerically indistinguishable there. Neither failure is fixed by tuning a threshold.

CCS CONCEPTS

• **Computing methodologies** → **Machine learning**; • **Mathematics of computing** → *Statistical paradigms*; • **Security and privacy** → *Software security engineering*.

KEYWORDS

target leakage, categorical data, Cramér's V, bias correction, missing data, data quality

Key insight. Two standard defaults—chi-squared bias on sparse categories, and complete-case deletion of missing feature values—each break categorical leakage screening in a different, diagnosable way. One is fixed by a bias-corrected estimator; the other requires treating missingness itself as a feature. Neither fix addresses the other's failure.

1 MOTIVATION

Standard categorical leakage screening thresholds an association statistic, usually Cramér's V, between each feature and the target. Throughout this paper, *leakage* means *target leakage* [1]: information about the label that enters training but would not be available at prediction time—not a security breach or data exfiltration. We show that on a single canonical dataset, this statistic fails in two directions for two distinct reasons, and that both reasons are properties of standard tooling defaults—not of V, and not idiosyncratic to our own implementation. The risk is not merely academic: near-unique identifier columns and administrative codes recorded post-outcome are common in exactly the security- and safety-relevant settings—fraud detection, credit scoring, clinical triage—where a missed leak is costly.

2 THE TWO FAILURES

Table 1 reports, for four Titanic columns, cardinality, singleton-category count, raw and bias-corrected Cramér's V, normalised mutual information $\widetilde{I}$, performance inflation π, the unified score $\mathcal{L}$ computed from each version of the correlation term ($\mathcal{L}(V) = 0.35V + 0.35\widetilde{I} + 0.30\pi$, and likewise for $\mathcal{L}(\widetilde{V})$), and the ground-truth leakage label. Reporting $\mathcal{L}$ under both statistics matters: under raw V, `name` and `sex` score almost identically ($\mathcal{L}$=0.407 vs. 0.405) despite one being leaky and the other clean; under corrected $\widetilde{V}$, `name` falls to $\mathcal{L}$=0.058 and the two are cleanly separated. `boat` reflects our pipeline *after* the fix described below.

Why `name` *inflates.* With 1,309 passengers and 1,307 distinct names, 1,305 names are singleton categories, and chi-squared-based association statistics are upward-biased whenever category counts are this sparse relative to sample size. The bias correction of Bergsma [2],

$$\widetilde{V} = \sqrt{\frac{\max\left(0,\ \varphi^2 - \frac{(r-1)(c-1)}{n-1}\right)}{\min(\tilde{r}-1,\ \tilde{c}-1)}}, \quad \tilde{r} = r - \frac{(r-1)^2}{n-1},\ \tilde{c} = c - \frac{(c-1)^2}{n-1},$$

collapses `name` to $\widetilde{V} = 0.000$—the estimator's clipped floor (the $\max(0, \cdot)$ term), not a coincidentally exact zero. `sex` and `embarked` barely move under the same correction, confirming it targets sparsity specifically.

Why `boat` *was missed, and by whom else.* Lifeboat assignment was recorded only for passengers whose fate was already known, making `boat` a textbook post-hoc leak, yet on our original pipeline it scored only $V = 0.420$. The reason is not that correlation is blind to this leak in principle: 62.9% of `boat` is missing, and that missingness is itself almost deterministic of the outcome (Table 2)—98.1% of passengers with a boat number survived, versus 2.8% of those without one. `pandas.crosstab` excludes rows with a missing value by default, and this is not a corner case we happened to hit: `ydata-profiling`'s own Cramér's V implementation calls `pandas.crosstab` directly, and empirically, Deepchecks' PPS-based `FeatureLabelCorrelation` scores `boat` at essentially zero (PPS

Table 1: Four Titanic columns, raw vs. bias-corrected Cramér's V, and the unified score computed from each. `name` is a false positive under raw V; `boat` (shown post-fix) is recovered from a false negative. †Clipped at the estimator's floor, not an exact zero.

Feature	Card.	Sing.	V	$\widetilde{V}$	$\widetilde{I}$	π	$\mathcal{L}(V)$	$\mathcal{L}(\widetilde{V})$	Truth
`name`	1307	1305	**0.999**	0.000†	0.164	0.000	0.407	**0.058**	clean (FP under V)
`boat` (post-fix)	28	4	0.951	0.940	1.000	0.808	**0.925**	0.922	leaky (recovered)
`sex`	2	0	0.529	0.528	0.265	0.424	0.405	0.405	clean
`embarked`	3	0	0.190	0.184	0.054	0.000	0.085	0.083	clean

= 0.0006) with real missing values present, versus 0.946 once missingness is encoded as an explicit indicator—the same failure, independently. This is a shared default across the tooling ecosystem, not an isolated implementation bug.

Table 2: Where the original `boat` signal was lost.

Comparison	n	V
`boat.notna()` vs. survived (full data)	1309	**0.948**
`boat` vs. survived (non-missing only)	486	0.420

We fixed this in our own pipeline: missing feature values are now encoded as an explicit category rather than dropped, matching how mutual information already treats them (`pandas.factorize` assigns missing values their own code rather than `NaN`, so our MI and performance terms never dropped this signal—only correlation did). Before → after: V(`boat`): 0.420 → 0.951; $\mathcal{L}$(`boat`): 0.739 → 0.925, crossing from *warning* into *error* severity.

3 WHY MULTIPLE SIGNALS HELP—AND WHY ONE FIX DOES NOT COVER BOTH FAILURES

Before the fix, $\widetilde{I}$ and π already saturated for `boat` while ρ did not (Table 1 reflects the fixed pipeline; pre-fix, $\mathcal{L}(V)$=0.739 and $\mathcal{L}(\widetilde{V})$=0.715, both still above the 0.7 warning threshold), because mutual information estimation [3] never dropped the missing rows to begin with. This is a real, useful property of combining signals, but it is not the fix for `boat`: the bias correction changes almost nothing for `boat`, before the fix (0.420 → 0.351) or after (0.951 → 0.940)—sparsity correction and missingness-aware association are two different remedies for two different failures, and applying one does not substitute for the other. The weights $(0.35, 0.35, 0.30)$ are a design choice, not a fitted parameter; we do not present the weighted sum itself as a research contribution.

4 NEGATIVE REPLICATION ON NSL-KDD

To test whether the sparsity mechanism generalises, we repeated the raw-vs.-corrected comparison on NSL-KDD [4] (125,973 rows), using its three native categorical features against a binary attack/normal label (46.5% attack rate). Table 3 reports the result: negative.

None of the three columns has any missing values, so this replication isolates the sparsity mechanism specifically. Even `service`, at 70 categories, has only 1 singleton among 125,973 rows—nothing like `name`'s 1,305 of 1,307—so the correction has nothing to remove. All three features are also strongly, legitimately associated with the attack label by design (network protocols and flags are diagnostic of specific attacks), so NSL-KDD offers no false-positive or false-negative case here; it confirms instead that the sparsity mechanism tracks observations-per-category, not cardinality by itself.

Table 3: NSL-KDD: raw and corrected V are indistinguishable at every cardinality tested—no column here is sparse enough to trigger bias.

Feature	Card.	Sing.	V	$\widetilde{V}$
`protocol_type`	3	0	0.282	0.282
`service`	70	1	0.860	0.860
`flag`	11	0	0.775	0.775

5 PRACTICAL IMPLICATIONS

Any feature that is a discrete, deterministic function of the outcome—assignment codes, post-event labels, administrative flags—can evade a thresholded association statistic while remaining fully visible to mutual information and single-feature performance. We recommend three concrete, independent practices: (1) use bias-corrected Cramér's V on high-cardinality columns; (2) screen missingness indicators as features in their own right, since informative missingness survives complete-case deletion in no standard tool we tested; (3) do not rely on a single statistic. Titanic is among the most widely used datasets in introductory machine-learning instruction, and the screening practice it is commonly used to illustrate is exactly the one shown insufficient here; introductory material should cover bias-corrected association measures and missingness screening alongside it. The bias-corrected estimator and the full open-source pipeline (349 automated tests) are available at https://github.com/jaimecano12/ml-framework.

REFERENCES

[1] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman. 2012. Leakage in data mining: Formalization, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data* 6, 4 (2012), 1–21.

[2] W. Bergsma. 2013. A bias-correction for Cramér's V and Tschuprow's T. *Journal of the Korean Statistical Society* 42, 3 (2013), 323–328. https://doi.org/10.1016/j.jkss.2012.10.002

[3] A. Kraskov, H. Stögbauer, and P. Grassberger. 2004. Estimating mutual information. *Physical Review E* 69, 6 (2004).

[4] M. Tavallaee, E. Bagheri, W. Lu, and A. Ghorbani. 2009. A detailed analysis of the KDD CUP 99 data set. In *Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*.